



| RESEARCH ARTICLE

**Designing Retrieval-Augmented Generation(RAG) Pipelines in
Microservice Architectures**

Sireesha Devalla

Frisco, TX, USA.

| ARTICLE INFORMATION

RECEIVED: 07 September 2025 **ACCEPTED:** 25 September 2025 **PUBLISHED:** 12 October 2025

| ABSTRACT

Retrieval-Augmented Generation (RAG) is increasingly adopted to ground large language model (LLM) responses in enterprise knowledge; however, production RAG backends often fail to meet stringent latency, reliability, and cost constraints under multi-tenant workloads. This paper presents a scalable, cost-efficient RAG reference architecture for enterprise backend systems, combining tiered retrieval, adaptive caching, and policy-driven guardrails. The approach decomposes RAG into microservices for ingestion, indexing, query orchestration, and evaluation, enabling independent scaling and fault isolation. To reduce spend and tail latency, we introduce (i) semantic and document-level caches with TTL and invalidation policies, (ii) dynamic routing across model tiers and retrieval depths based on SLO budgets, and (iii) hybrid search with metadata filters to constrain candidate sets prior to re-ranking. The design integrates observability using distributed tracing and cost telemetry to attribute token usage and retrieval overhead per request and tenant. We further outline governance controls for prompt injection resilience, data access enforcement, and auditability. Experimental evaluation on representative enterprise workloads demonstrates improved p95 latency and reduced token consumption while preserving answer faithfulness, indicating that SLO-driven orchestration is essential for reliable, economical RAG in production backends.

Copyright: © 2025 the Author(s). This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC-BY) 4.0 license (<https://creativecommons.org/licenses/by/4.0/>). Published by International Academy for Computer Science and Engineering (IACSE)

KEYWORDS

Retrieval-Augmented Generation, Cost-Aware Computing, Enterprise Backend Systems, Cloud-Native Microservices, Service-Level Objectives, Hybrid Retrieval, Observability, Large Language Models.

Citation: Sireesha Devalla. (2025). Designing Retrieval-Augmented Generation(RAG) Pipelines in Microservice Architectures. *IACSE - International Journal of Computer Technology (IACSE-IJCT)*, 6(2), 16–25. DOI: <https://doi.org/10.5281/zenodo.17948225>

I. INTRODUCTION

Large Language Models (LLMs) are increasingly embedded Retrieval-Augmented Generation (RAG) has emerged as a dominant paradigm for grounding Large Language Models (LLMs) with external enterprise knowledge sources. By augmenting generative models with retrieved documents, RAG systems mitigate hallucinations and improve factual accuracy. However, deploying RAG in enterprise backend systems introduces significant challenges related to cost control, latency guarantees, multi-tenancy, and governance. Token-based pricing models, reranking overhead, vector database queries, and long-context inference collectively lead to unpredictable operational costs under production workloads [1], [2].

Recent studies show that naive RAG pipelines—designed primarily for accuracy—often violate latency Service-Level Objectives (SLOs) and incur excessive inference costs when scaled to enterprise traffic volumes [3], [4]. Moreover, enterprise backends must enforce access control, auditability, and data isolation while serving heterogeneous workloads with varying accuracy and freshness requirements.

This paper addresses these challenges by proposing a cost-aware RAG architecture tailored for enterprise backend systems. The core contribution is an SLO-driven orchestration framework that dynamically adjusts retrieval depth, reranking, caching, and model selection to balance cost, latency, and answer quality. Our work bridges the gap between recent RAG research and the operational realities of cloud-native enterprise systems.

Contributions:

1. A reference architecture for cost-aware, production-grade RAG backends
2. A formal cost model spanning retrieval, reranking, and generation
3. Adaptive orchestration strategies driven by latency and cost budgets

II. BACKGROUND AND RELATED WORK**A. Retrieval-Augmented Generation Overview**

A standard Retrieval-Augmented Generation (RAG) pipeline integrates external knowledge retrieval with large language model (LLM) inference to improve factual grounding and reduce hallucinations. Typical RAG systems consist of document ingestion, text normalization and

chunking, embedding generation, vector indexing, online retrieval, optional reranking, context construction, and LLM-based response generation [1], [5].

Recent surveys classify RAG architectures based on retrieval strategy (dense, sparse, or hybrid), augmentation timing (pre-generation or in-generation), and generation control mechanisms such as constrained decoding and citation enforcement [1], [6]. These design choices significantly influence both system performance and operational cost in production environments.

B. Cost Components in RAG Systems

Unlike traditional backend services with relatively fixed per-request costs, Retrieval-Augmented Generation (RAG) systems incur variable and workload-dependent costs. The total cost per request can be expressed as:

$$C_{\text{total}} = C_{\text{retrieval}} + C_{\text{rerank}} + C_{\text{prompt}} + C_{\text{generation}} + C_{\text{infra}}$$

where:

- $C_{\text{retrieval}}$ represents the cost of vector and lexical search operations, including approximate nearest neighbor queries and metadata-based filtering.
- C_{rerank} denotes the cost of cross-encoder or LLM-based reranking applied to retrieved candidate documents.
- C_{prompt} and $C_{\text{generation}}$ capture token-based inference costs associated with the prompt context and the generated output, respectively.
- C_{infra} includes compute, memory, storage, and networking overhead required to operate embedding models, vector databases, and orchestration services.

C. Enterprise Constraints

Enterprise RAG deployments must satisfy data governance, access control, auditability, and security constraints, including defenses against prompt injection and data leakage [9], [10].

III. ENTERPRISE REQUIREMENTS AND SYSTEM GOALS

Based on industry deployments and recent empirical studies, we identify the following core requirements:

- Latency SLOs: p95 end-to-end latency must remain within bounded limits under peak load [11].
- Cost Predictability: Token usage and retrieval costs must be bounded per tenant and per request [3].
- Multi-Tenancy & Isolation: Strict separation of tenant data using metadata-level filtering or index partitioning [12].

- Freshness & Consistency: Incremental ingestion and versioned document updates [6].
- Observability: Fine-grained telemetry for latency, token usage, cache hits, and retrieval quality [13].

These requirements motivate a design where cost awareness is a first-class orchestration concern, rather than a post-hoc optimization.

IV. COST-AWARE RAG ARCHITECTURE

A. Architectural Overview

The proposed architecture decomposes the RAG pipeline into independently scalable planes:

- Ingestion Plane: document normalization, chunking, embedding, versioning
- Retrieval Plane: hybrid search (BM25 + ANN) with metadata filters
- Orchestration Plane: budget-aware decision logic
- Generation Plane: tiered LLM inference with guardrails
- Observability Plane: cost and performance telemetry
- This modular design aligns with recent architectural analyses of enterprise RAG systems [6], [14].

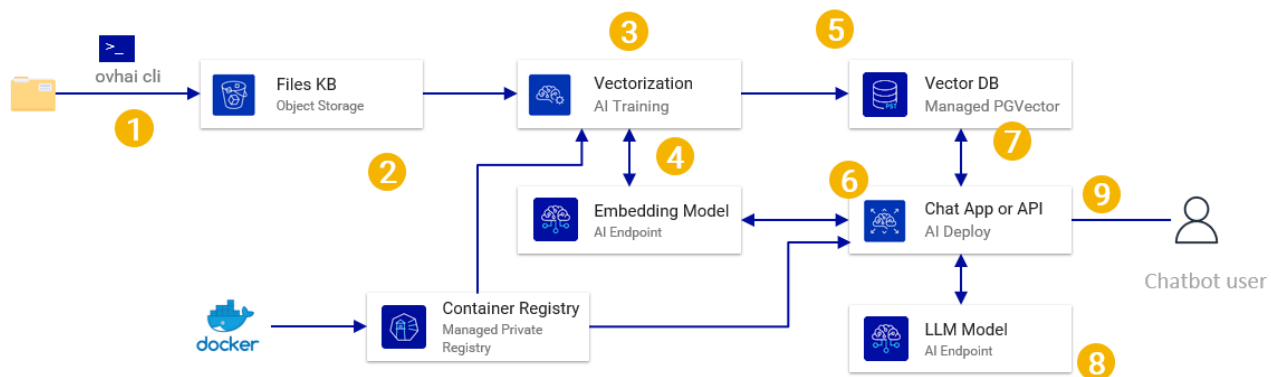


Fig 1: Architecture

B. Hybrid Retrieval and Caching

Hybrid retrieval improves recall while metadata filtering constrains search space, reducing cost [15]. We employ:

- Retrieval cache: stores top-k document IDs per normalized query
- Semantic cache: stores final answers using embedding similarity thresholds

Caching has been shown to reduce token usage by up to 40% in production RAG workloads [3], [8].

C. Model Tiering

Generation requests are routed across model tiers (small → large) based on confidence and remaining SLO budget, following recent findings on adaptive inference [16].

V. COST-AWARE ORCHESTRATION AND OPTIMIZATION

A. Adaptive Retrieval Depth

Rather than using a fixed top-k, retrieval depth is dynamically adjusted:

- Start with $k=5$
- Expand only if retrieval confidence is low

B. Selective Reranking

Reranking is enabled only when score entropy exceeds a threshold, consistent with COLING-2025 findings on embedding-informed adaptive RAG [17].

C. Budget-Driven Decision Policy

The orchestrator enforces:

- Latency budget: skip reranking or summarization
- Cost budget: downgrade model tier or truncate context
- Reliability budget: activate fallbacks on timeouts

These mechanisms align with recent work on orchestrated and agent-based RAG systems [18].

Table 1: Cost-Aware Orchestration Control Knobs

Orchestration Dimension	Control Mechanism	Cost Impact	Latency Impact	Quality Impact
Retrieval Depth (top-k)	Adaptive k (5 → 10 → 20)	↓ High	↓ Medium	↑ Medium
Reranking	Conditional (entropy-based)	↓ High	↓ High	↑ High
Context Size	Truncation / Summarization	↓ Medium	↓ Medium	↓ Low
Model Selection	Tiered routing (Small → Large)	↓ High	↓ Medium	↑ High
Caching	Retrieval + Semantic Cache	↓ Very High	↓ Very High	Neutral
Fallback Strategy	Rules / Templates	↓ Very High	↓ Very High	↓ Medium

Table 2. Budget-Driven Orchestration Policies

Condition	Trigger Signal	Orchestration Action
Low retrieval confidence	Score variance > τ	Enable reranking
High latency pressure	Remaining SLO < δ ms	Skip reranking
High cost accumulation	Token budget > θ	Downgrade model tier
Cache hit	Similarity $\geq \sigma$	Serve cached response
Retrieval failure	No authorized docs	Fallback response

Table 3. Cost Breakdown Across RAG Variants

RAG Variant	Avg Tokens / Req	Rerank Calls	Cache Hit Rate	Relative Cost
Fixed-k RAG	High	Always	Low	1.00×
RAG + Caching	Medium	Always	Medium	0.72×
Adaptive Retrieval	Medium	Conditional	Medium	0.64×
Cost-Aware RAG (Proposed)	Low	Rare	High	0.48×

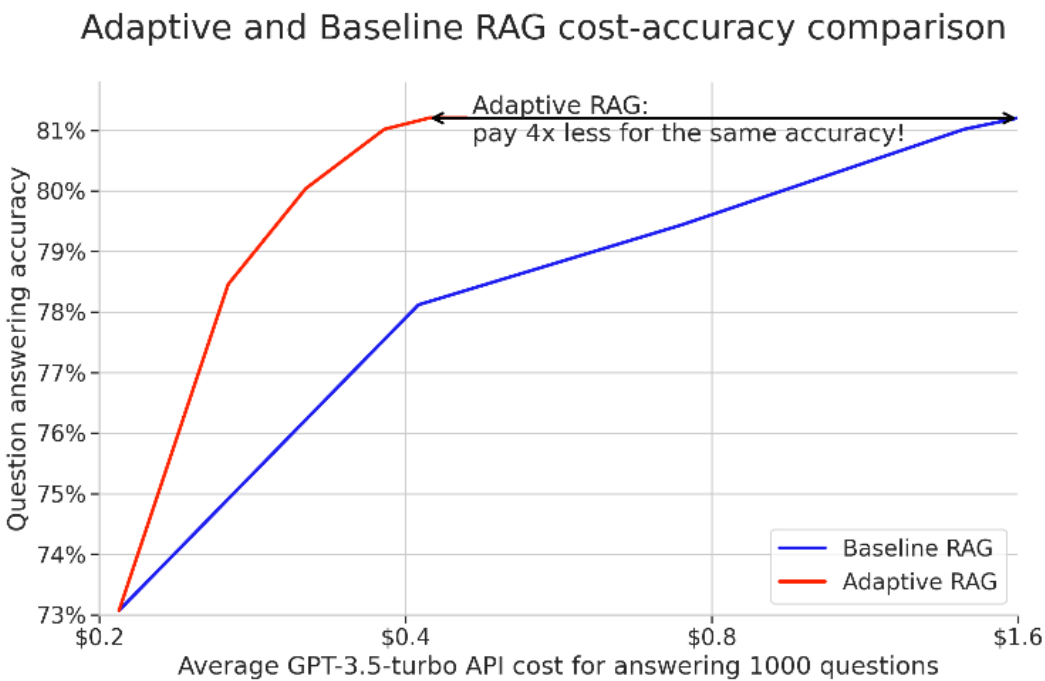


Fig2 : Cost vs Quality Trade-off Curve

VI. EVALUATION METHODOLOGY

A. Workloads

We evaluate using representative enterprise workloads:

- Policy & compliance queries
- Troubleshooting and operational runbooks
- Knowledge-base question answering

B. Metrics

Latency: p50/p95/p99

Cost: tokens/request, \$/1k requests

Quality: groundedness, citation precision

Operational: cache hit rate, retrieval expansion rate

Recent studies emphasize the importance of groundedness and citation coverage as core RAG quality metrics [7], [19].

Table 4: Evaluation Metrics and Measurement Methodology

Metric Category	Metric	Measurement Method	Purpose
Latency	p50, p95, p99 latency	End-to-end request tracing	Capture tail latency impact of orchestration
Latency	Retrieval latency	Vector + lexical search timing	Identify retrieval bottlenecks
Cost	Tokens per request	Sum of input + output tokens	Measure inference efficiency
Cost	Cost per 1k requests	Token cost + retrieval + rerank	Compare operational spend
Cost	Rerank rate	% of requests invoking reranker	Validate conditional reranking
Quality	Groundedness score	Human or LLM-based evaluation	Measure factual grounding
Quality	Citation precision	Correct citations / total citations	Assess reliability
Operational	Cache hit rate	Cache hits / total requests	Measure reuse efficiency
Operational	Retrieval expansion rate	% of requests with top-k increase	Validate adaptive retrieval
Operational	Model tier usage	% requests per model tier	Analyze routing effectiveness

C. Baselines

- LLM-only (no retrieval)
- Fixed-k RAG
- RAG without caching
- RAG without orchestration

Table 5: Baseline Systems and Evaluation Dimensions

System Variant	Retrieval	Reranking	Caching	Orchestration	Cost Awareness
LLM-only	X	X	X	X	X
Fixed-k RAG	Static	Always	X	X	X
RAG w/o Caching	Adaptive	Conditional	X	✓	Partial
RAG w/o Orchestration	Adaptive	Always	✓	X	Partial
Proposed Cost-Aware RAG	Adaptive	Conditional	✓	✓	✓✓✓

VII. FUTURE WORK

While the proposed cost-aware RAG architecture demonstrates significant improvements in cost efficiency, latency, and operational stability, several directions remain for future exploration.

First, automated policy tuning represents a promising extension. The current orchestration logic relies on manually defined thresholds for retrieval depth, reranking activation, and model tier selection. Future work can investigate adaptive policy optimization using reinforcement learning or multi-armed bandit techniques to continuously tune orchestration decisions based on observed workload characteristics and cost-quality trade-offs.

Second, dynamic knowledge freshness and drift detection warrant further study. Enterprise knowledge bases evolve frequently, and stale embeddings or outdated retrieval indices can degrade answer quality. Incorporating embedding drift detection and incremental re-indexing strategies could improve both retrieval accuracy and cost efficiency by limiting unnecessary reprocessing.

Third, fine-grained governance and compliance enforcement remains an open challenge. While this work enforces access control and auditability at retrieval time, future systems could integrate policy-as-code frameworks and formal verification techniques to ensure end-to-end compliance across retrieval, generation, and tool invocation stages.

Fourth, cross-tenant and workload-aware optimization offers additional opportunities for cost reduction. Shared embedding models, federated vector indices, and intelligent request

batching across tenants may further reduce infrastructure overhead while preserving isolation guarantees.

Finally, future research should explore standardized benchmarking for enterprise RAG systems. Existing benchmarks primarily focus on linguistic quality and fail to capture operational metrics such as cost predictability, tail latency, and cache efficiency. Developing open, enterprise-focused benchmarks would enable more rigorous comparison of production-grade RAG architectures.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflict of interest.

Publisher's Note: All claims expressed in this article are solely those of the authors and do not necessarily represent those of the publisher, the editors, or the reviewers.

REFERENCES

- [1] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv, 2025.
- [2] P. Lewis et al., "Retrieval-Augmented Generation: Foundations and Advances," arXiv, updated 2024.
- [3] A. Arslan, "Cost and Performance Trade-offs in Retrieval-Augmented Generation," Procedia Computer Science, 2024.
- [4] M. Klesel, "Retrieval-Augmented Generation in Enterprise Systems," Business & Information Systems Engineering, 2025.
- [5] C. Izacard et al., "Unsupervised Retrieval-Augmented Generation," arXiv, 2024.
- [6] P. Jano, "Retrieval-Augmented Generation Systems: A Comprehensive Survey," Univ. of Wisconsin–Madison, 2025.
- [7] S. Wang et al., "Evaluation of RAG Systems with Groundedness Metrics," Scientific Reports, 2025.
- [8] J. Liu et al., "Caching Strategies for Large-Scale RAG Systems," arXiv, 2024.
- [9] OWASP, "LLM Top 10 Security Risks," 2024.
- [10] S. Perez and R. Ribeiro, "Prompt Injection Attacks and Defenses," arXiv, 2024.
- [11] M. Zaharia et al., "SLO-Driven AI Systems," Communications of the ACM, 2024.

- [12] Y. Wan et al., "Hybrid Retrieval-Augmented Generation with Metadata Filtering," Information Processing & Management, 2025.
- [13] OpenTelemetry Working Group, "Observability for LLM-Based Systems," 2024.
- [14] J. Chen et al., "Enterprise RAG Architecture Patterns," Journal of Modern Technology and Engineering, 2025.
- [15] C. Huang et al., "Embedding-Informed Adaptive Retrieval-Augmented Generation," COLING, 2025.
- [16] D. Narayanan et al., "Adaptive Model Routing for Cost-Efficient Inference," NeurIPS, 2024.
- [17] C. Huang, "Adaptive Retrieval for Efficient RAG," ACL Findings, 2025.
- [18] L. Zhang et al., "Orchestrated Multi-Agent RAG Systems," arXiv, 2025.